

使用深度摄像头与激光雷达获取道路信息数据

一、任务描述

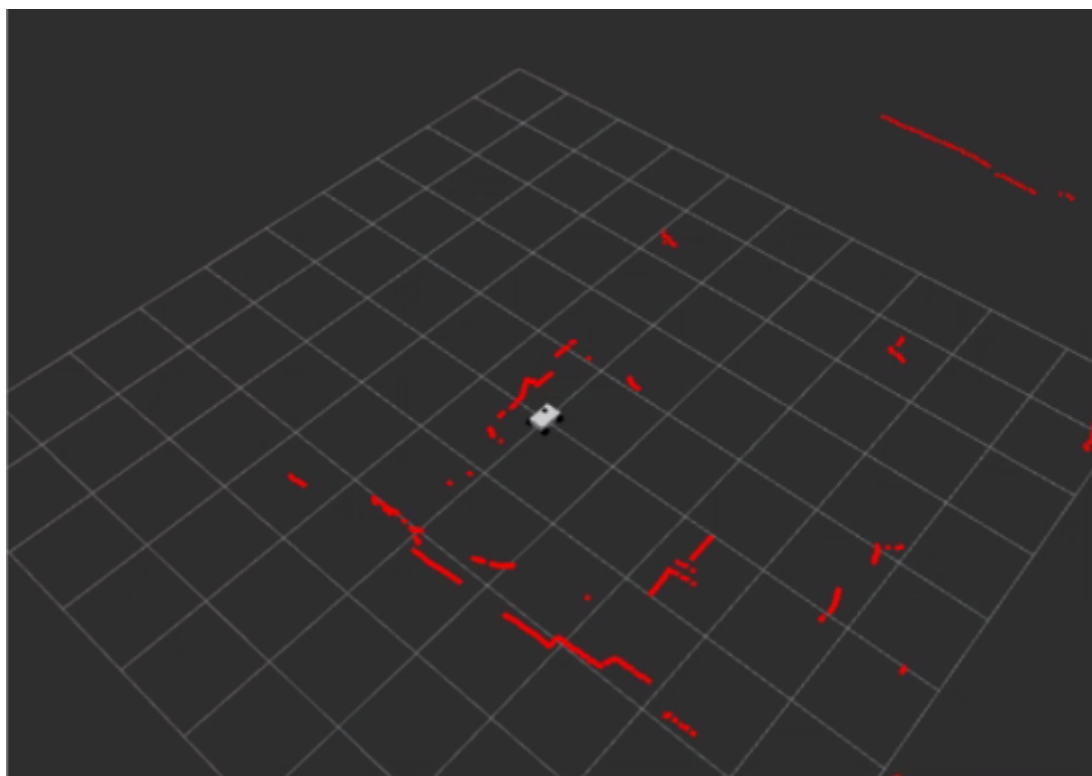
激光雷达是实现自动驾驶汽车的一项关键技术，但是，激光雷达并不能看到一切，深度相机将填补激光雷达的探测盲区。

尽管目前有各种先进的技术为自动驾驶服务，但是自动驾驶汽车在某些重要方面仍然存在盲区。激光雷达（Light Detection and Ranging）是一种通过发射不可见光波段的激光脉冲，计算其从物体上反射回来的时间，以确定物体距离的方法。激光雷达作为实现自动驾驶的一种使能技术，受到了广泛关注。虽然激光雷达并不是一种全新的技术，但许多人认为它是实现自动驾驶的关键。

激光雷达提供了即时定位和地图构建（SLAM）功能，它解决了几乎不可能实现的挑战，即使车辆高速移动，也能近乎实时地测绘未知环境的地图，并知道车辆新环境中的位置。有了SLAM功能，自动驾驶汽车将拥有正常行驶所需要的远程信息。但是，自动驾驶还有其他挑战需要克服。

精确探测临近区域对于成功开发自动驾驶汽车至关重要。深度学习技术可以教会3D系统以高保真度感知和识别对象。激光雷达可以实现SLAM和导航，深度相机可以确保障碍物的感知和识别。

本次实训任务，我们将用自动驾驶汽车上的激光雷达和深度摄像头，来获取道路（环境）数据，并且实现这些数据的可视化。



二、任务目标

- 了解深度摄像头的测距原理。
- 掌握获取深度摄像头数据的方法。
- 了解激光雷达测距原理。

- 掌握获取激光雷达数据的方法

三、任务环境

- 操作系统: Ubuntu20.04 ROS-neotic
- 工具软件: NoMachine
- 必备硬件: 自动驾驶汽车

四、任务原理

4.1 认识深度相机 (TOF)

4.1.1 深度相机(TOF)的工作原理

TOF (Time of flight) 直译为“飞行时间”。其测距原理是通过给目标连续发送光脉冲, 然后用传感器接收从物体返回的光, 通过探测光脉冲的飞行 (往返) 时间来得到目标物距离。这种技术跟3D激光传感器原理基本类似, 只不过3D激光传感器是逐点扫描, 而TOF相机则是同时得到整幅图像的深度 (距离) 信息。

4.1.2 深度相机(TOF)的组成

1、照射单元

照射单元需要对光源进行脉冲调制之后再进行发射, 调制的光脉冲频率可以高达100MHz。因此, 在图像拍摄过程中, 光源会打开和关闭几千次。各个光脉冲只有几纳秒的时长。相机的曝光时间参数决定了每次成像的脉冲数。

要实现精确测量, 必须精确地控制光脉冲, 使其具有完全相同的持续时间、上升时间和下降时间。因为即使很小的只是一纳秒的偏差即可产生高达15 c m的距离测量误差。

如此高的调制频率和精度只有采用精良的LED或激光二极管才能实现。

一般照射光源都是采用人眼不可见的红外光源。

2、光学透镜

用于汇聚反射光线, 在光学传感器上成像。不过与普通光学镜头不同的是这里需要加一个带通滤光片来保证只有与照明光源波长相同的光才能进入。这样做的目的是抑制非相干光源减少噪声, 同时防止感光传感器因外部光线干扰而过度曝光。

3、成像传感器

TOF的相机的核心。该传感器结构与普通图像传感器类似, 但比图像传感器更复杂, 它包含2个或者更多快门, 用来在不同时间采样反射光线。因此, TOF芯片像素比一般图像传感器像素尺寸要大得多, 一般100um左右。

4、控制单元

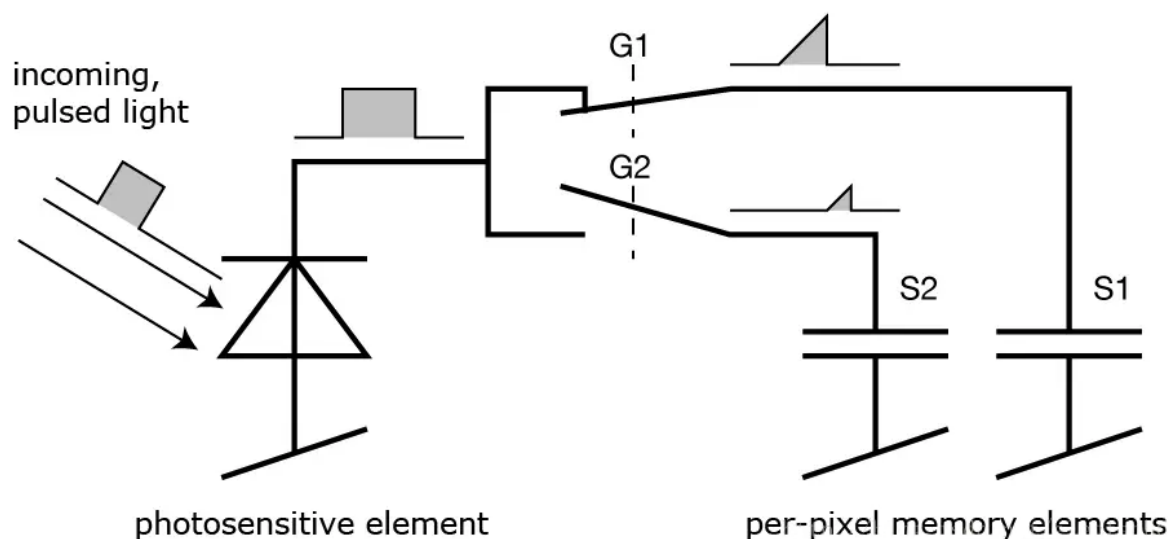
相机的电子控制单元触发的光脉冲序列与芯片电子快门的开/闭精确同步。它对传感器电荷执行读出和转换, 并将它们引导至分析单元和数据接口。

5、计算单元

计算单元可以记录精确的深度图。深度图通常是灰度图, 其中的每个值代表光反射表面和相机之间的距离。为了得到更好的效果, 通常会进行数据校准。

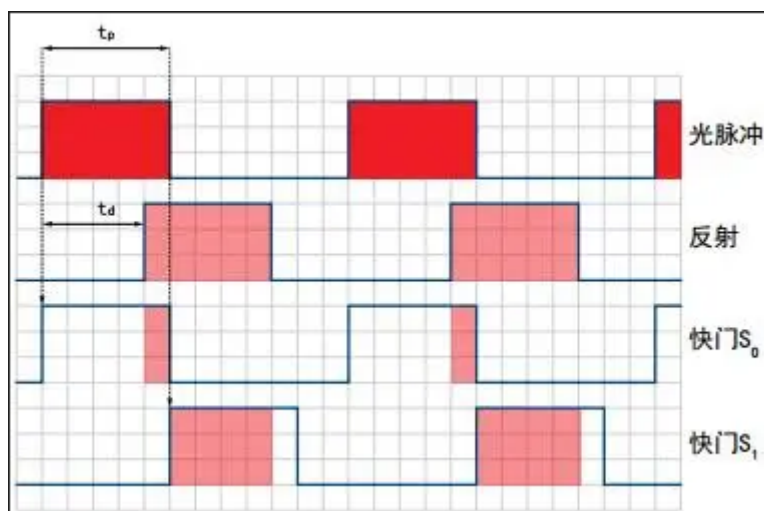
4.1.3 深度相机(TOF)如何测距

照射光源一般采用方波脉冲调制，这是因为它用数字电路来实现相对容易。深度相机的每个像素都是由一个感光单元（如光电二极管）组成，它可以将入射光转换为电流，感光单元连接着多个高频转换开关（下图的G1，G2）可以把电流导入不同的可以储存电荷(下图S1，S2)的电容器里。



相机上的控制单元打开光源然后再关闭，发出一个光脉冲。在同一时刻，控制单元打开和关闭芯片上的电子快门。由光脉冲以这种方式产生的电荷S0存储在感光元件上。

然后，控制单元第二次打开并关闭光源。这次快门打开时间较晚，即在光源被关闭的时间点打开。现在生成的电荷S1也被存储在感光元件上。



因为单个光脉冲的持续时间非常短，此过程会重复几千次，直到达到曝光时间。然后感光传感器中的值会被读出，实际距离可以根据这些值来计算。

记光的速度为c， t_p 为光脉冲的持续时间，S0表示较早的快门收集的电荷，S1表示延迟的快门收集的电荷，那么距离d可以由如下公式计算：

$$d = \frac{c}{2} \times t_p \times \frac{S_1}{S_0 + S_1}$$

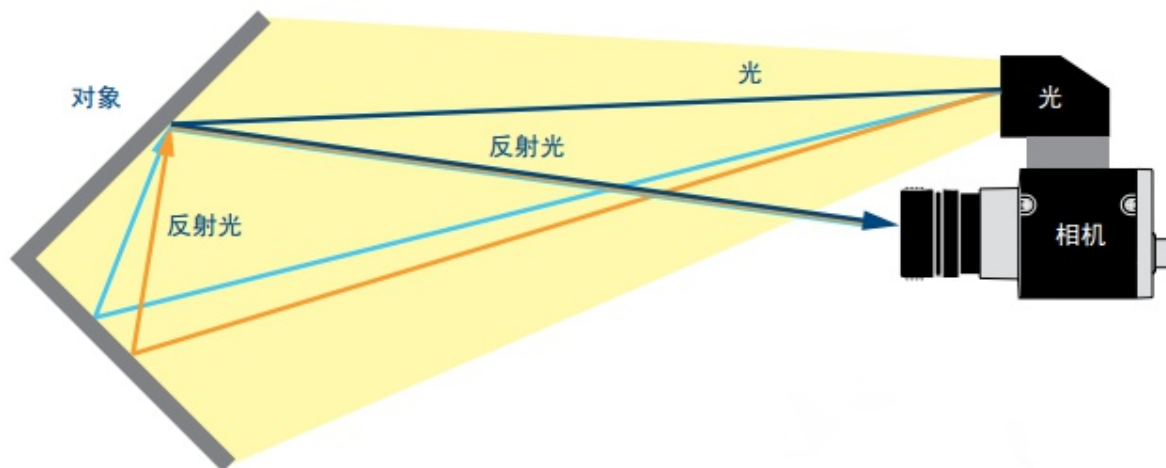
最小的可测量距离是：在较早的快门期间S0中收集了所有的电荷，而在延迟的快门期间S1没有收集到电荷，即 $S_1 = 0$ 。代入公式会得出最小可测量距离 $d=0$ 。

最大的可测量的距离是：在S1中收集了所有电荷，而在S0中根本没有收集到电荷。然后，该公式得出 $d = 0.5 \times c \times t_p$ 。因此最大可测量距离是通过光脉冲宽度来确定的。例如， $t_p = 50 \text{ ns}$ ，代入上式，得到最大测量距离 $d = 7.5 \text{ m}$ 。

4.1.4 TOF会受什么影响

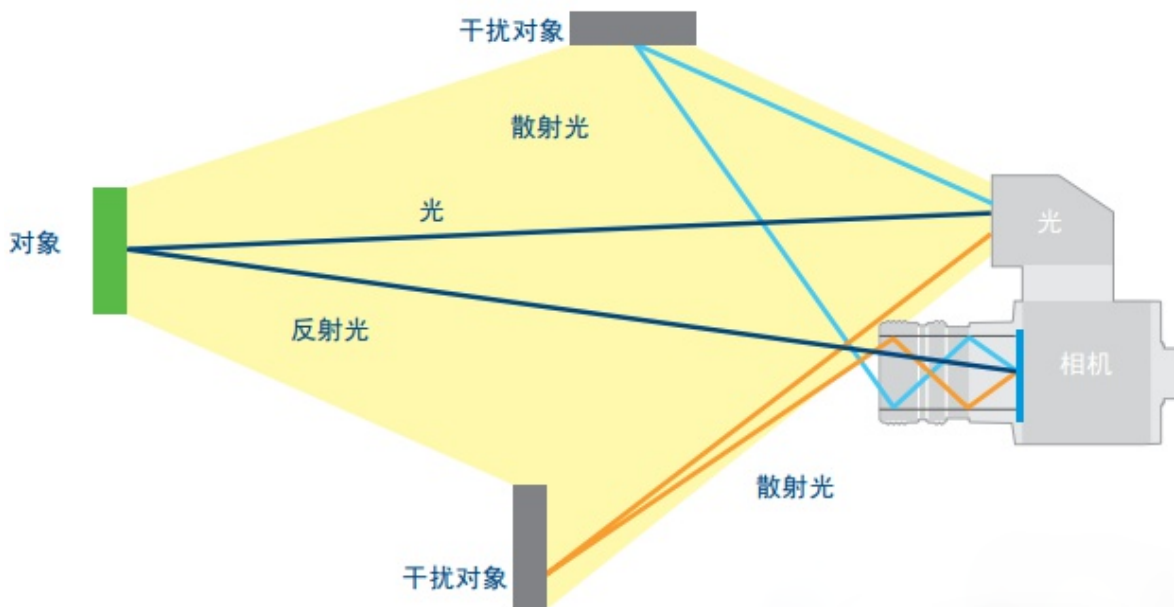
1、多重反射。

距离测量要求光只反射一次。但是镜面或者一些角落会导致光线的多次反射，这会导致测量失真。如果多重反射使得光线完全偏转，则没有反射光线进入相机，从而无法正确测量反射面的距离。反之，如果其他方向的光通过镜面反射进入芯片，则可能会发生过度曝光。见下图。



2、散射光

在镜头内或在镜头后面发生多余反射会出现散射光，如下图所示，散射光会导致图像褪色，对比度下降等不良影响。所以要避免在相机正前方有强烈反光的物体存在。



3、环境光

前面说过，深度相机镜头上会有一个带通滤光片来保证只有与照明光源波长相同的光才能进入，这样可以抑制非相干光源提高信噪比。这种方式确实能够比较有效地过滤掉人造光源，但是，我们常见的日光几乎能够覆盖整个光谱范围，这其中包括和照明光源一样的波长，在某些情况下（如夏天的烈日）这部分光强可以达到很大，会导致感光传感器出现过度曝光。因此相机如果想在這種条件下正常工作，仍然需要额外的保护机制。

4、温度

电子元件的精度受温度的影响。所以当温度波动时会影响电子元件的性能，从而影响到脉冲调制的精度。前面说过一纳秒的脉冲偏差即可产生高达15 c m的距离测量误差，因此相机要做好散热，这样才能保证测量精度。

4.1.5 深度相机输出

TOF相机内部每个像素经过上述过程都可以得到一个对应的距离，所有的像素点测量的距离就构成了一幅深度图，如下图所示。左边是原图，右边是对应的深度图。

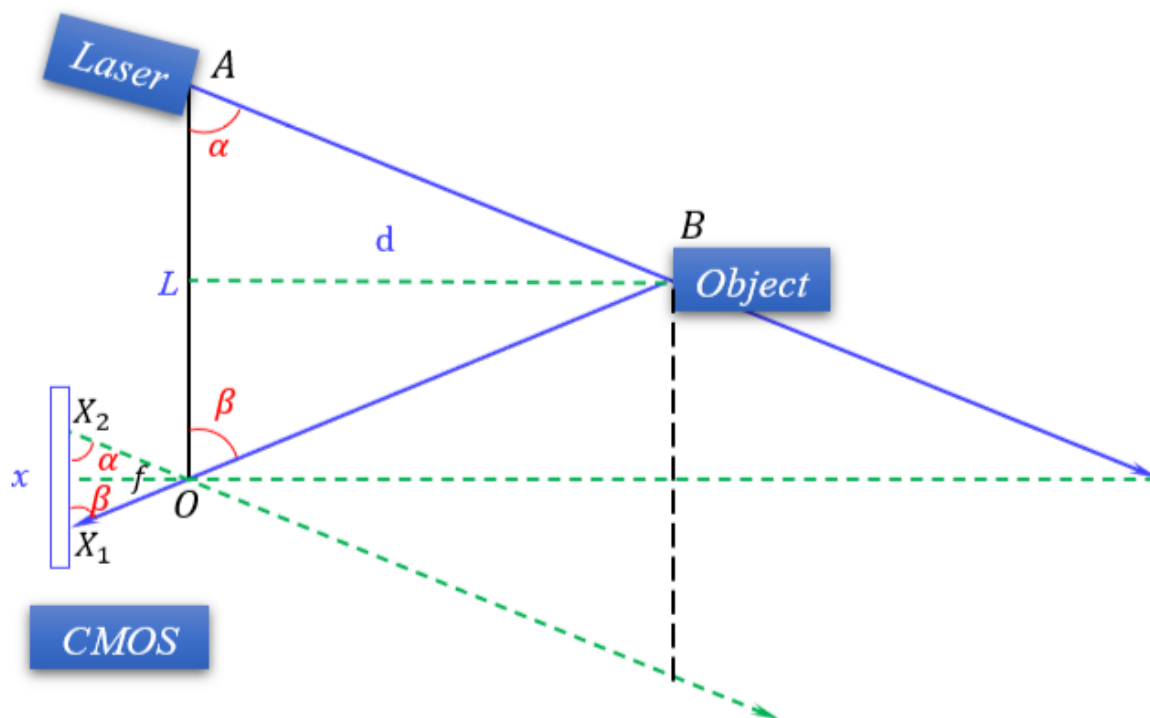


可以看到深度图其实是一张灰度图，它是一张三维的图：水平垂直坐标对应像素位置，该位置的灰度值对应的是该像素距离摄像头的距离。所以深度图中的每个像素可以表示空间中一个点的三维坐标，所以深度图中的每个像素也称为体像素（voxel）。

4.2 激光雷达传感器测距原理

常见的激光雷达主要有三角测距和TOF两大类型。

4.2.1 三角测距



如图所示，为三角测距激光雷达的原理图。首先激光发射器（Laser）发射激光，打到物体（Object）表面时，将反射至CMOS相机处，经过相机焦点与图像交于X1。

经过相机焦点O做激光的平行线交相机成像于X2处，X1 X2 间距离为x，相机焦距为f。易得 $\triangle OAB \sim \triangle X1X2O$ ，则可以得到测量值d的表达式：

$$\frac{L}{x} = \frac{d}{f}$$

$$d = f \frac{L}{x}$$

式中，f为CMOS相机焦距大小；L为激光发射器与CMOS相机间距；参量x可由三角函数得到：

$$x = \frac{f}{\tan \alpha} + \frac{f}{\tan \beta}$$

$$x = f \frac{\sin(\alpha + \beta)}{\sin \alpha \sin \beta}$$

其中，α 为激光发射器发射角度，角度β 可由相机内参矩阵计算得到。将x带入测量值表达式，进行化简：

$$d = L \frac{\sin \alpha \sin \beta}{\sin(\alpha + \beta)}$$

三角测距法参量表如下：

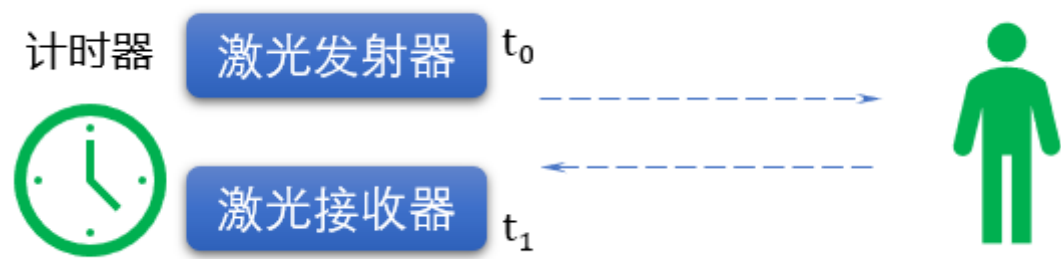
参量	含义
α	激光发射角，标定后固定
β	相机接收角，
L	激光发射器与相机间距离，标定后固定
f	相机焦距，固定值
d	激光测距值
x	中间变量，图像上X ₁ X ₂ 间距离

采用三角测距原理制成的激光雷达传感器具有以下特点：

- 1 成本低、价格便宜
- 2 中近距离精度较高
- 3 远距离精度较差
- 4 易受干扰，一般用于室内定位

当测量物体较远，即d值较大时，所引起的x变化较小，精度较差。

4.2.2 TOF测距



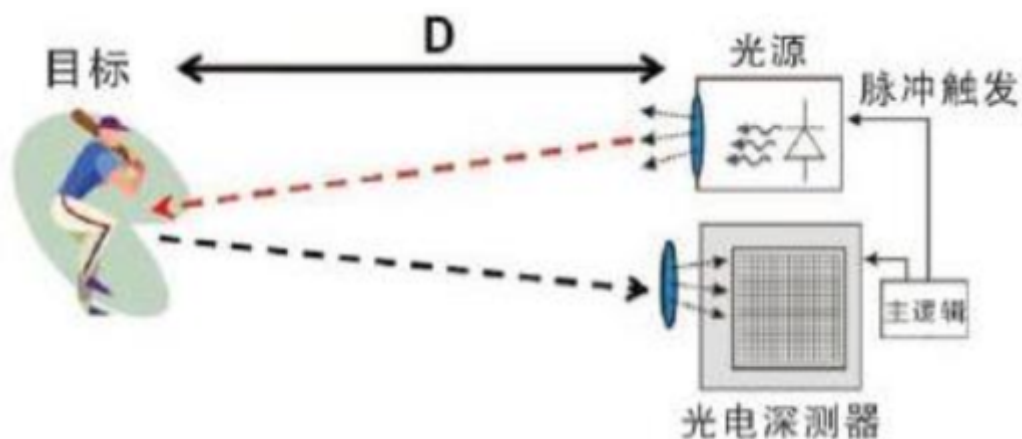
飞行时间 (Time of flight) 利用光速进行测距，若激光发射器在 t_0 时刻发射一束激光，打到物体上并反射，在 t_1 时刻激光接收器接收到，则测量距离可用如下表达式计算：

$$D = C \frac{(t_1 - t_0)}{2}$$

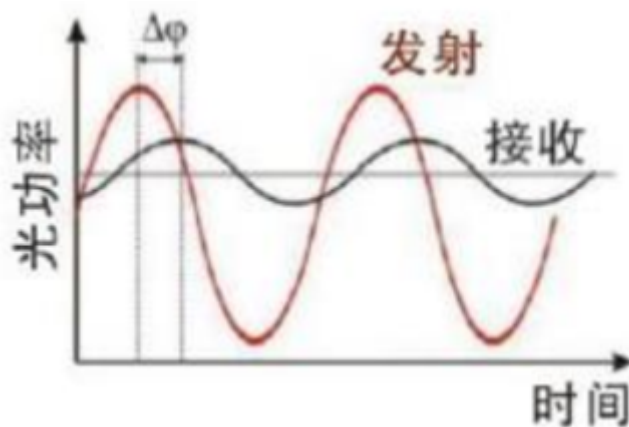
式中， C 表示光速 $3 \times 10^8 \text{ m/s}$ 。

由于光速过快，对计时器精度要求高，测量精度1m对应时间精度ns；测量精度cm则对应时间精度ps。

故而，在实际应用中通常采用测量型号相位代替直接测量时间，如下图所示。



TOF时间



波形相位差

则时间可用相位差进行计算：

$$\Delta t = \frac{\Delta \phi}{2\pi f_m}$$

$$D = C \frac{\Delta t}{2} = C \frac{\Delta \phi}{4\pi f_m}$$

时间测距的参量表如下：

参量	含义
t_0	激光发射时间
t_1	激光接收时间
C	光速 $3 \times 10^8 \text{m/s}$
$\Delta\varphi$	激光收发波形相位差
D	激光测距值
f_m	一个周期的频率

采用飞行时间原理制成的激光雷达传感器特点如下：

- 1 价格昂贵
- 2 中远距离精度高、近距离较差
- 3 测距范围广
- 4 抗干扰能力强，可用于室外定位

当测量物体较近，即D值较小时，由于对计时器精度较高，故而测量精度一般。

五、任务步骤

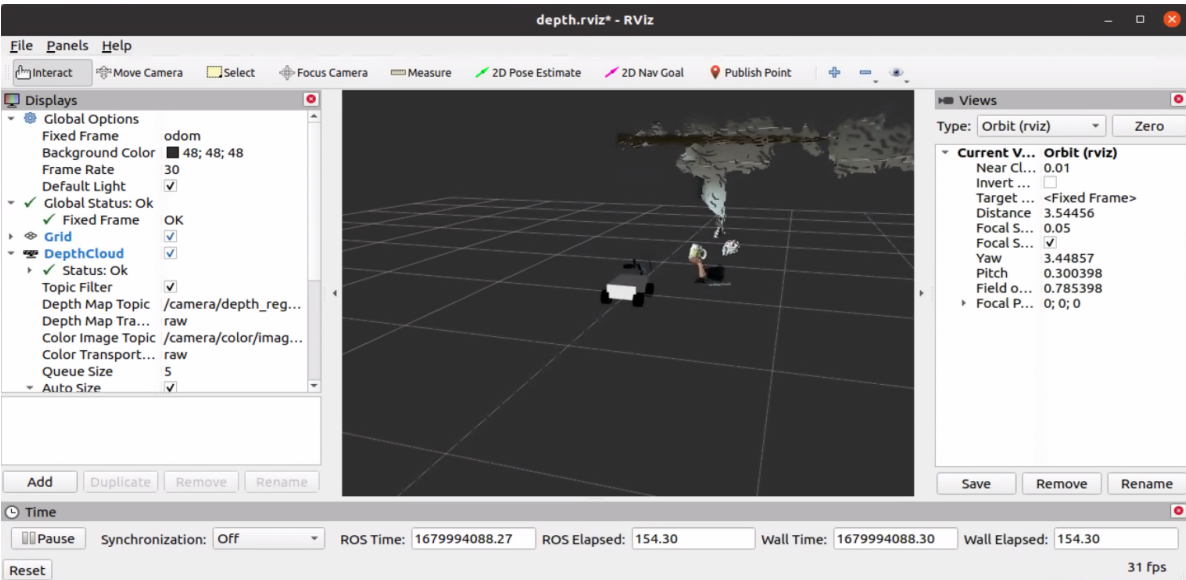
步骤一 获取深度摄像头数据

1.1 RVIZ显示深度图像

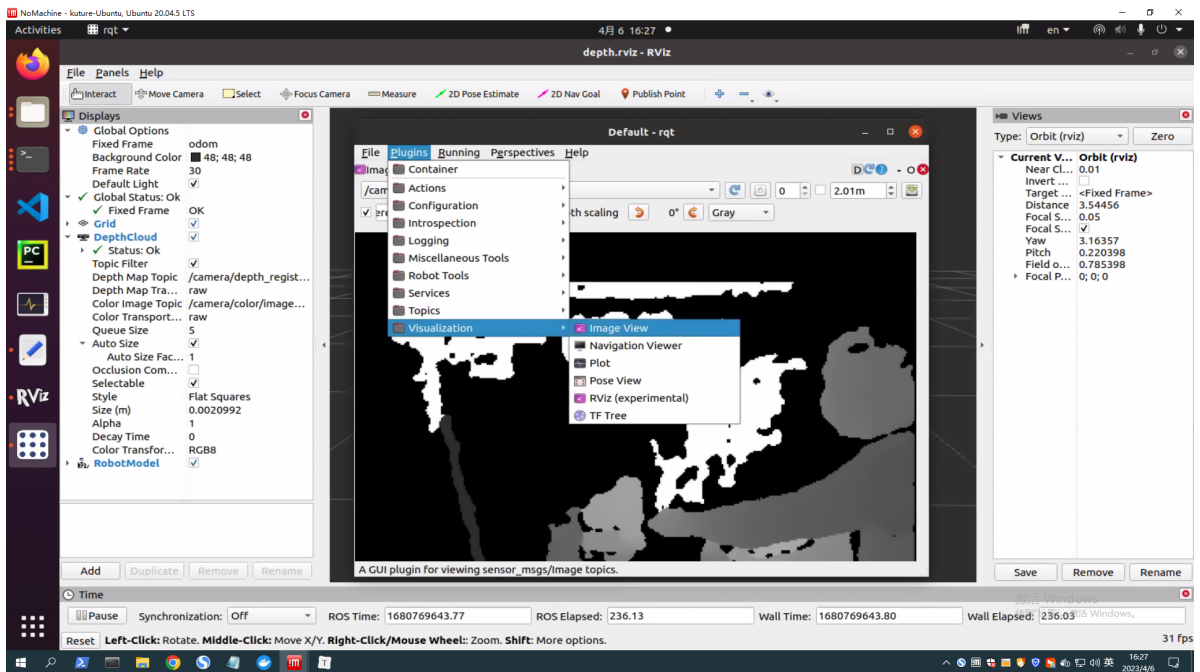
打开终端，进入目录“/home/mo/ros_learning_demos/sample_shell”，执行如下代码：

```
sh chapter_3_depth_cam.sh
```

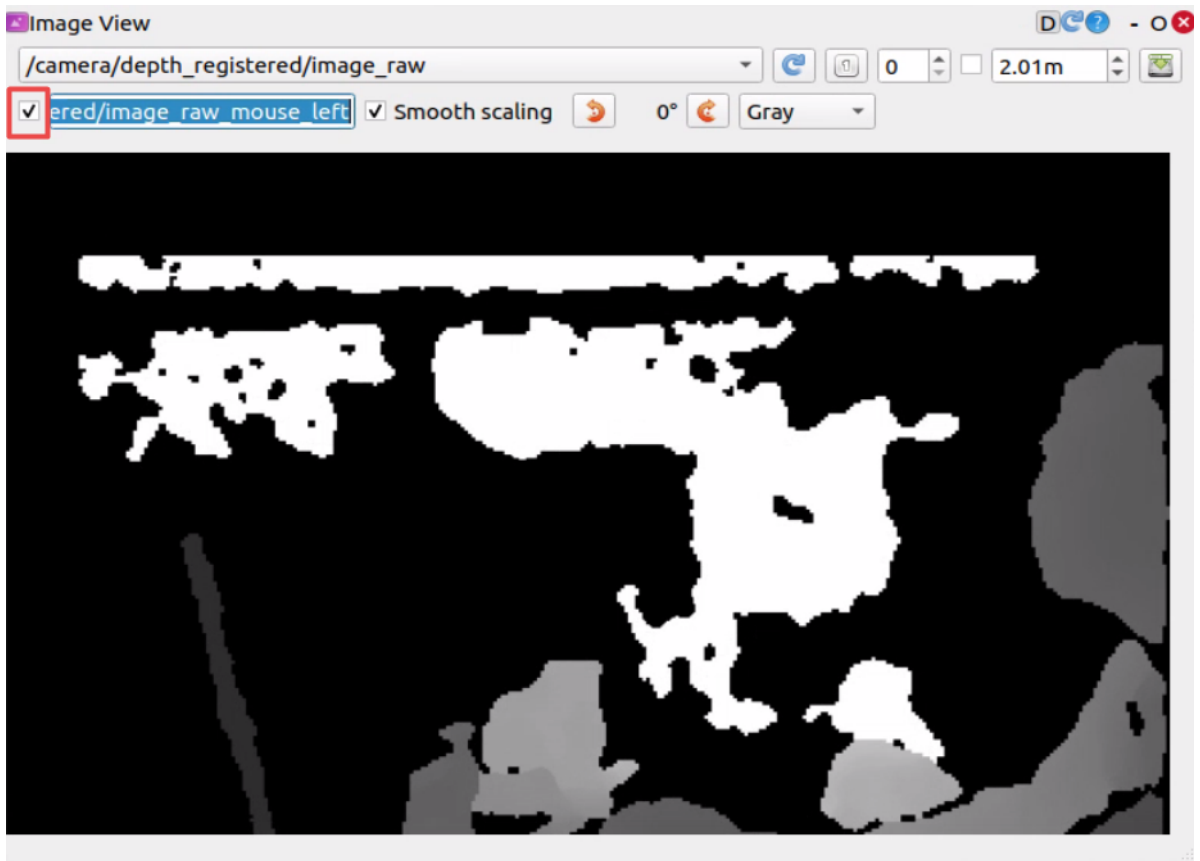
打开RVIZ显示小车深度摄像头视角：



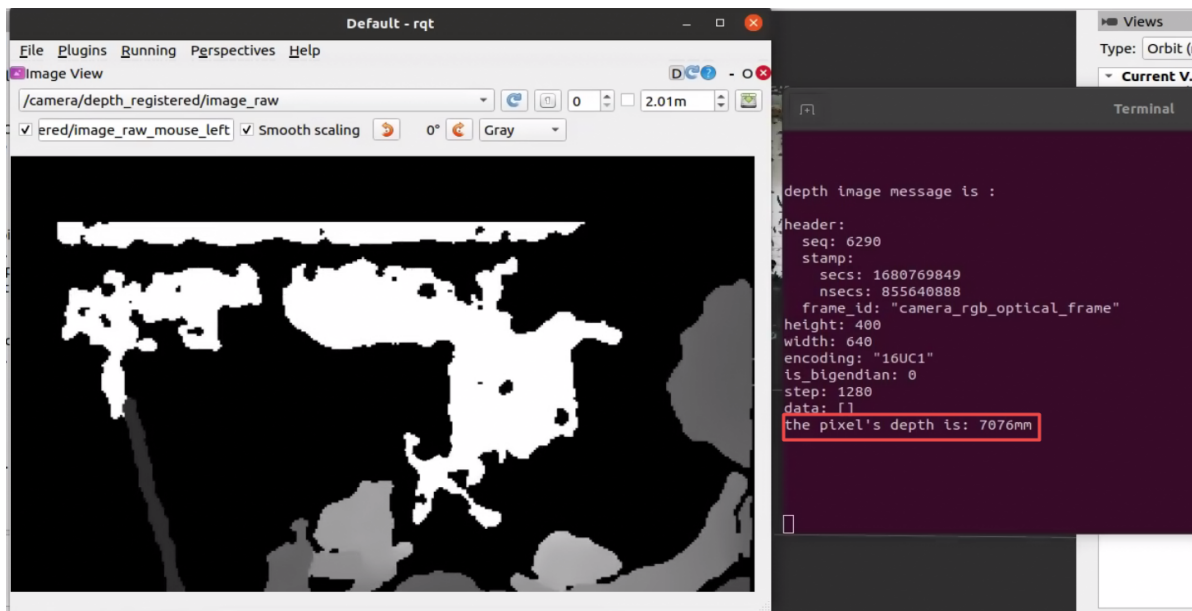
在打开的rqt中，选择“Plugins”→“Visualization”→“Image View”，可以打开视觉画面。



勾选"/camera/depth_registered/image_raw_mouse_left"前面的对勾。



鼠标单击图像区域，可以在终端显示点击像素点的深度信息：



1.2 代码实现深度图像数据的获取

创建文件“/home/mo/ros_learning_demos/sample_shell/chapter_3_depth_cam.sh”，添加如下内容：

```
#!/bin/bash
#source /opt/ros/melodic/setup.bash
#source ~/ros_space/devel/setup.bash

gnome-terminal "启动深度摄像头" -- bash -c " roslaunch learning_sensor
depth_camera.launch;killall depth_msg_reader rqt"
sleep 1
gnome-terminal "消息可视化界面" -- bash -c " rqt;killall roslaunch
depth_msg_reader"
sleep 4
gnome-terminal "深度消息解析" -- bash -c " rosrn learning_sensor
depth_msg_reader.py;killall roslaunch rqt"
```

此节点用来调用depth_camera.launch节点，打开rqt，打开消息可视化界面，调用深度信息获取脚本：depth_msg_reader.py。

创建文

件“/home/mo/ros_learning_demos/src/learning_ros/learning_sensor/launch/depth_camera.launch”，添加如下内容，调用深度摄像头节点和RVIZ节点：

```
<launch>
<!-- 底盘与深度摄像头API -->
<include file="$(find ai_node_master)/cfg/API_cfg.launch" >
  <arg name="encoder_only"          default="true" />
</include>

<!-- 可视化界面 -->
<node pkg="rviz" type="rviz" name="rviz" args="-d $(find
learning_sensor)/rviz/depth.rviz" output="screen" required="true"/>
</launch>
```

这里调用了API_cfg.launch节点，创建"/home/mo/ros_space/src/ai_node_master/cfg/API_cfg.launch"，添加如下内容，实现深度摄像头的调用：

```
<launch>
  <!--参数配置-->
  <arg name="load_depth_driver" default="false" />      <!--读取深度硬件-->
  <arg name="load_rgb_driver" default="false" />         <!--读取RGB摄像头硬件-->
  >
  <arg name="encoder_only" default="false" />            <!--禁用激光雷达矫正-->
  <arg name="driver_node" default="true" />              <!--开启ROS自带底盘节点控制-->

  <arg name="use_API" default="true" />                  <!--使用API-->
  <arg name="IMU_msg" default="false" />                 <!--API获取IMU信息-->
  <arg name="driver_msg" default="false" />              <!--API获取底盘信息-->
  <arg name="msg_debug" default="false" />              <!--打印debug-->

  <!--API读取节点-->
  <node if="$(arg use_API)" name="ros_communicator" pkg="ai_node_master"
type="ros_communicator.py" output="screen" >
    <param name="depth_stream_flag" value="$(arg load_depth_driver)" />
    <param name="rgb_stream_flag" value="$(arg load_rgb_driver)" />
    <param name="IMU_msg" value="$(arg IMU_msg)" />
    <param name="driver_msg" value="$(arg driver_msg)" />
    <param name="msg_debug" value="$(arg msg_debug)" />
  </node>

  <!--底盘控制节点-->
  <include file="$(find ai_node_master)/launch/base_driver.launch" >
    <arg name="use_API" value="$(arg use_API)" />
    <arg name="driver_node" value="$(arg driver_node)" />
    <arg name="encoder_only" value="$(arg encoder_only)" />
  </include>

  <!--深度摄像头节点-->
  <include file="$(find astra_camera)/launch/gemini.launch" >
    <arg name="load_depth_driver" default="$(arg load_depth_driver)" />
    <arg name="load_rgb_driver" default="$(arg load_rgb_driver)" />
  </include>

</launch>
```

此处实现深度摄像头显示的节点是"/home/mo/ros_space/src/astra_camera/launch/gemini.launch"，此节点为系统节点。

创建文

件"/home/mo/ros_learning_demos/src/learning_ros/learning_sensor/scripts/depth_msg_reader.py"，输入如下内容，实现深度信息的获取：

```
#!/usr/bin/python3
```

```

import rospy
import cv2
import cv_bridge
from geometry_msgs.msg import Point
from sensor_msgs.msg import Image
import numpy as np
import threading
from copy import deepcopy

# 初始化节点
rospy.init_node('depth_msg_reader', anonymous=True)
roster = 0

# cv_bridge将深度消息转化为cv::mat格式
cvbr = cv_bridge.CvBridge()

# 鼠标左键点击处的坐标
pixel = [0,0]

# 深度信息处理
def depth_msg_callback(img):
    global pixel, roster
    if pixel[0]*pixel[1] != 0 :

        # 打印深度数据，因data数据量太大，这里在打印时将其实际数据删除
        if rospy.Time().now().to_sec() - roster > 1:
            data = img.data
            img.data = []
            print('depth image message is :\n')
            print(img)

            img.data = data

        # 转换为cv::mat格式
        depth_img = cvbr.imgmsg_to_cv2(img, 'passthrough')

        # 打印出鼠标左键点击处的深度数据值，单位为mm
        print("the pixel's depth is: " + str(depth_img[pixel[1], pixel[0]]))
+'''mm

        '''
        roster = rospy.Time().now().to_sec()

# 鼠标坐标处理
def pixel_msg_callback(msg):
    global pixel
    pixel[0] = int(msg.x)
    pixel[1] = int(msg.y)

def pixel_listener():
    print('Listening to mouse left')
    while not rospy.is_shutdown():
        # 订阅鼠标左键点击的坐标
        rospy.Subscriber('/camera/depth_registered/image_raw_mouse_left', Point,
pixel_msg_callback)

```

```

if __name__ == '__main__':

    # 新增订阅鼠标左键线程
    pixel_listen = threading.Thread(target=pixel_listener)
    pixel_listen.start()

    print('Listening to depth message')
    rospy.Subscriber('/camera/depth_registered/image_raw', Image,
depth_msg_callback)
    rospy.spin()

```

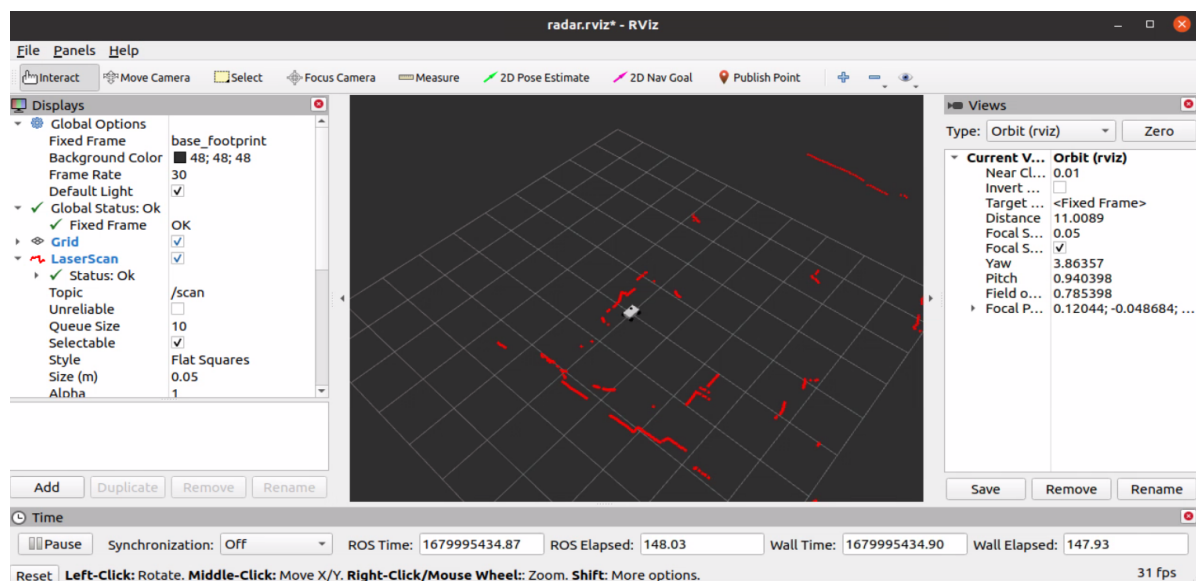
步骤二 获取激光雷达数据

2.1 RVIZ显示激光雷达图像

打开终端，进入目录"/home/mo/ros_learning_demos/sample_shell"，执行如下代码：

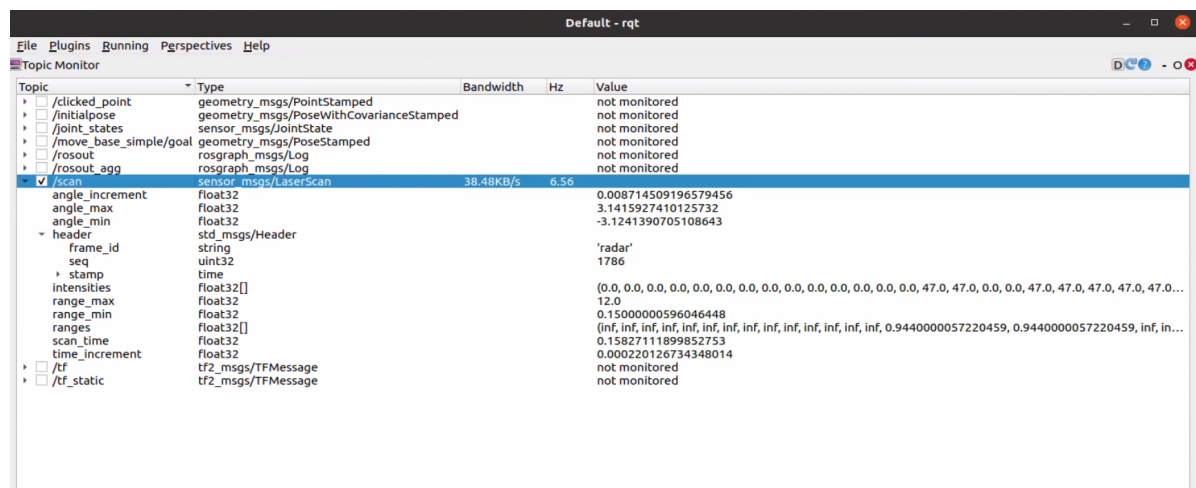
```
sh chapter_3_radar.sh
```

打开RVIZ，显示激光雷达获取到的数据图像：



其中红色边缘是激光雷达检测到的障碍物。

再打开一个终端，输入命令"rqt"，打开rqt界面，选择"/scan"，可以查看激光雷达返回的数据：



激光雷达数据结构解析：

- Header :是一个结构体，包含seq、stamp、frame—id
 - seq扫描顺序增加的id序列
 - stamp激光数据的时间戳
 - frame-id是扫描数据的名字
- angle-min：开始扫描的角度
- angle-max：结束扫描的角度
- angle-increment：每次扫描增加的角度
- time-increment：测量的时间间隔
- scan-time：扫描的时间间隔
- range-min：测距的最小值
- range-max：测距的最大值
- ranges：转一周的测量数据一共360个
- intensities：与设备有关，强度数组长度360

2.2 代码实现激光雷达数据的获取

创建文件"/home/mo/ros_learning_demos/sample_shell/chapter_3_radar.sh"，添加如下内容，实现激光雷达节点的调用：

```
#!/bin/bash
# source /opt/ros/noetic/setup.bash
# source ~/ros_learning_demos/devel/setup.bash
gnome-terminal "启动talker" -- bash -c " roslaunch learning_sensor
radar.launch;killall roslaunch "
```

此处调用radar.launch节点。创建节点文件"/home/mo/ros_learning_demos/src/learning_ros/learning_sensor/launch/radar.launch"，添加如下内容：

```
<launch>
  <include file="$(find rplidar_ros)/launch/rplidar.launch" />
  <include file="$(find urdf_car)launch/unionws_test.launch"/>
  <node pkg="rviz" type="rviz" name="rviz" args="-d $(find
learning_sensor)/rviz/radar.rviz" output="screen" required="true"/>
</launch>
```

此节点调用rplidar.launch节点和RVIZ显示节点。rplidar.launch节点为系统节点。

六、任务总结

自动驾驶汽车的最终成功将取决于可靠性、经过测试、经济高效且万无一失的视觉系统。解决所有这些需求的最安全的解决方案是激光雷达和3D视觉的结合。这两种技术的结合，能够立即识别车辆的位置及其附近的物体，以在几分之一秒内做出必要的反应，以避免交通困难甚至灾难。同时，他们将确保驾驶员在必要时能够随时准备好接管车辆的驾驶任务。目前，自动驾驶仍然存在着某些视觉问题，但是3D视觉和激光雷达这两种技术的结合，将为自动驾驶克服视觉盲区提供有远见的解决方案。

